Report 2: Routy

Rodrigo Perestrelo, Lennard Last
ID2201 Distributed Systems, Basic Course

September 16, 2026

1 Introduction

The objective of this assignment is to design, implement, and empirically evaluate a decentralized link-state routing protocol. This protocol requires every router to independently construct an identical global graph of the network topology and locally compute optimal next-hop forwarding paths using Dijkstra's algorithm.

2 What Was Difficult and How It Was Solved

2.1 Implementing Dijkstra's Algorithm and Routing Table Construction

The Problem: Implementing Dijkstra's algorithm in a purely functional language required iteratively relaxing paths while keeping track of the direct next-hop gateway for every destination.

The Solution: In `update/4`, we check whether a node already exists in the sorted list via `entry/2`. If absent (returning `0`), it has already been settled and is safely discarded. If present with a greater distance, it is replaced with the shorter path and the corresponding gateway:

```
update(Node, N, Gateway, Sorted) ->
    case entry(Node, Sorted) of
        0 -> Sorted;
        Length when Length > N ->
            replace(Node, N, Gateway, Sorted);
        Length when Length =< N ->
            Sorted
    end.
```

During table generation in `iterate/3`, the node with the minimal distance is extracted from the sorted list, committed to the final routing table as `{Dest, Gateway}`, and its adjacent links from `Map` are relaxed using `update/4` before recursing.

2.2 Preventing Broadcast Storms and Stale LSAs in Cyclic Networks

The Problem: Link-state advertisements flooded over cyclic topologies will loop indefinitely if routers lack memory of forwarded packets, causing broadcast storms and packet duplication.

The Solution: In `hist.erl`, each router tags advertisements with an incrementing integer sequence counter N. A received message from router $Node$ with sequence number R is discarded as `old` if $R \leq$ Counter. Furthermore, by initializing a router's own identity as $\{Name, \text{inf}\}$, any self-generated message that cycles back through the network is rejected:

```
update(Node, N, History) ->
    case lists:keyfind(Node, 1, History) of
        false -> {new, [{Node, N} | History]};
        {Node, Counter} when Counter < N ->
```

```
            NewHist = lists:keydelete(Node, 1, History),
            {new, [{Node, N} | NewHist]};
        {Node, Counter} when Counter >= N -> old
    end.
```

3 Potential Improvements to Routy

1. **Automated Event-Driven Flooding and Re-convergence:** Currently, link-state flooding (`broadcast`) and path recomputation (`update`) are triggered manually via console messages. Routy should automatically increment its sequence counter and flood an updated LSA immediately upon receiving a `'DOWN'` monitor signal, followed by a scheduled Dijkstra execution.

2. **Weighted Edge Costs:** The current protocol assumes unit link costs (hop count). Incorporating latency or bandwidth metrics into `map.erl` would allow Dijkstra to select throughput-optimal paths.

3. **LSA Garbage Collection:** The history tracking list in `hist.erl` grows indefinitely as new nodes are discovered. Implementing a time-to-live (TTL) on history records would prevent memory leaks in large topologies.

4 Automated Testing and Bonus Evaluation: The World

To demonstrate Routy's functionality and evaluate the questions for the bonus assignment, we implemented an automated test harness (`world_test.erl`) that creates an 8-node intercontinental network across four regions:

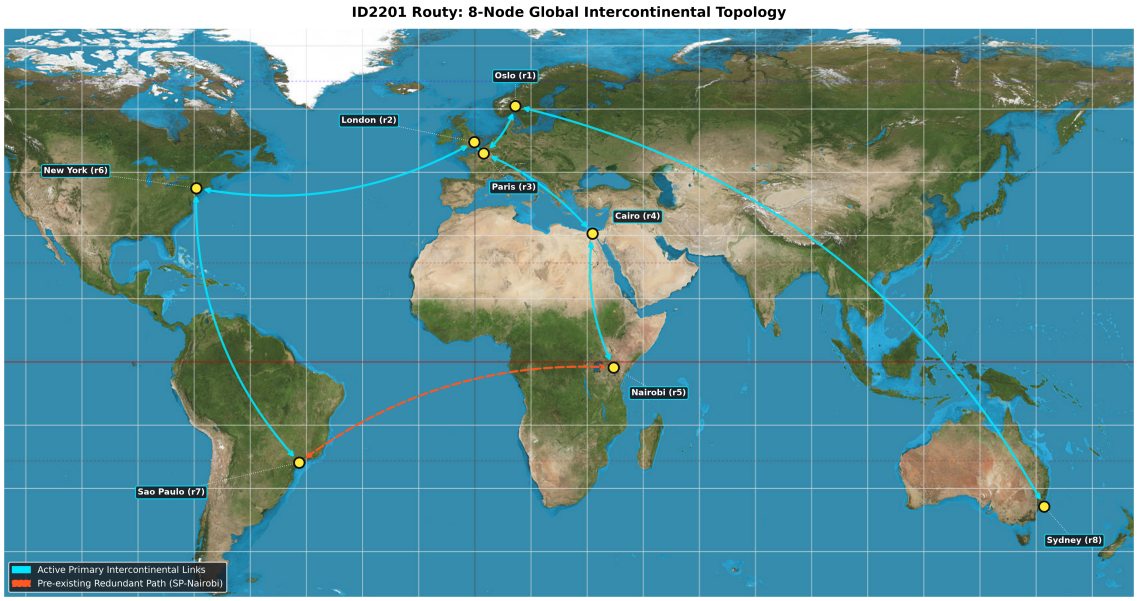


Figure 1: Geographical deployment of the 8-node Routy network across 4 continents. Solid cyan links designate active primary paths; the dashed orange link indicates the pre-existing redundant link between São Paulo and Nairobi.

4.1 Can we send messages from Sydney to Oslo?

Yes. Once link states flooded through the network and routing tables were constructed, intercontinental forwarding succeeded directly across regions:

```
sydney:  routing message ("Prompt test:  Direct intercontinental message Sydney to Oslo")
oslo:  received message "Prompt test:  Direct intercontinental message Sydney to Oslo"
```

Furthermore, multi-hop transatlantic traffic from Sao Paulo to Sydney successfully traversed North America and Europe via London:

```
sao_paulo → new_york → london → paris → oslo → sydney
```

4.2 Will routing functionality still work if we stop routers or close nodes?

Yes, provided an alternate physical path exists in the topology and routers re-converge.

When London (`r2`) was terminated (`routy:stop(r2)`), its direct peers (Paris and New York) instantly caught the termination via Erlang process monitors without crashing:

```
paris:  exit recived from london
new_york:  exit recived from london
```

Upon running a broadcast and update cycle on surviving nodes, the network re-converged autonomously around pre-existing backup links. Retransmitting from Sao Paulo to Sydney successfully detoured through Africa without manual intervention:

```
sao_paulo → nairobi → cairo → paris → oslo → sydney
```

4.3 How long between a disabled network card and the delivery of a 'DOWN' message?

The detection latency depends directly on the failure mechanism:

- **Router Process Crash or Shell Exit (`q().`):** Takes < 2 **ms** to < 15 **ms**. When a process dies or an OS terminates the VM, the operating system kernel closes open TCP sockets immediately with a `FIN` or `RST` packet. The remote Erlang runtime traps this socket closure instantly.
- **Disabled Network Interface / Pulled Cable:** Takes **45 to 60 seconds**. When a physical link is severed, no TCP teardown packets are transmitted. Surviving nodes cannot distinguish an idle connection from a dead interface. The runtime must wait for Erlang's distribution heartbeat timer (`net_ticktime`, default 60 seconds) to expire before flagging the node as unreachable and dispatching the `'DOWN'` message.

5 Conclusions

This laboratory demonstrated the separation between the control plane (graph flooding and shortest-path computation) and the data plane (packet forwarding). Erlang's lightweight actor model, process monitors, and pattern matching provided clean abstractions for implementing link-state protocols. The experiments proved that sequence numbers are necessary to eliminate broadcast storms, and revealed that detecting silent physical network partitions requires tuning heartbeat intervals.